



Introduction au développement Web

J-2

Récapitulatif & Transition : de la structure au style

Avant de plonger dans l'univers passionnant du **CSS**, faisons un rapide retour sur les bases vues lors du **jour 1**. Comprendre le rôle de **HTML** en tant que fournisseur de contenu est essentiel pour apprécier toute la puissance du CSS.



À quoi sert HTML ?

HTML (HyperText Markup Language) est la colonne vertébrale des pages web.

Il sert à **structurer le contenu**, à en définir le sens et à fournir une **hiérarchie logique**.

👉 On peut le comparer au **plan** et aux **matériaux bruts** d'un bâtiment.

2

Quelles balises avez-vous utilisées dans votre mini CV ?

Hier, vous avez expérimenté différentes balises comme :

- `<h1>` → titres,
- `<p>` → paragraphes,
- `<ul>` / `<li>` → listes,
- `<img>` → images,
- `<a>` → liens.

Ces balises définissent sémantiquement les différentes parties de votre contenu.

Transition: **HTML** construit le squelette de votre page.

Aujourd'hui, nous allons lui donner de la **vie**, de la **personnalité** et un **attrait visuel** grâce au **CSS**.

👉 Imaginez un bâtiment magnifiquement conçu, avec des détails raffinés, des couleurs harmonieuses et un aménagement paysager soigné : c'est exactement ce que **CSS** apporte à votre **HTML** !

Principe fondamental : la séparation des responsabilités

L'un des concepts les plus puissants du développement web est la **claire séparation entre le contenu et le style**.

- **HTML** se concentre uniquement sur la **structure** et le **sens** de votre contenu.
- **CSS**, quant à lui, est dédié à sa **présentation** et à sa **mise en page**.

HTML: Contenu & Structure

HTML est le langage de balisage qui fournit le contenu brut (texte, images, vidéos) et définit sa **structure logique**.

Il indique au navigateur : « *Ceci est un titre* », « *Ceci est un paragraphe* » ou « *Ceci est une image* ».

👉 Il se concentre sur **ce que le contenu est**.

CSS: Style & Apparence

CSS (Cascading Style Sheets) est le langage de style qui dicte comment les éléments HTML sont affichés.

Il contrôle les **couleurs**, les **polices**, les **espacements**, le **positionnement** et l'ensemble du **design visuel**.

👉 CSS rend votre HTML brut **beau et agréable à utiliser**.



Code plus clair

Séparer HTML et CSS permet d'avoir un code plus **organisé** et **lisible**.

- Les fichiers HTML restent **sémantiques**.
- Les fichiers CSS se concentrent uniquement sur le **style**.

👉 Résultat : un développement et un débogage beaucoup plus simples.



Maintenance plus Facile

Si vous devez changer l'apparence de votre site, il suffit de modifier le fichier **CSS**.

C'est bien plus efficace que de parcourir le HTML pour mettre à jour des styles intégrés.



Style réutilisable

Un seul fichier **CSS** peut styliser plusieurs pages HTML.

Cela garantit une **cohérence visuelle** sur tout votre site et vous fait gagner beaucoup de temps, car vous définissez vos styles une seule fois pour les appliquer partout.



Demo: `<h1>Hello World</h1>`. Avec un fichier CSS, on peut le rendre **rouge et centré**.

En changeant simplement de fichier CSS, on peut l'avoir **bleu et aligné à gauche**, sans toucher au HTML.

Le fonctionnement du CSS

CSS applique des règles aux éléments HTML.
Chaque règle se compose de :

- un **sélecteur** → indique quel élément HTML cibler,
- un **bloc de déclaration** → contient une ou plusieurs déclarations.

CSS Rule Structure

```
sélecteur {  
  propriété: valeur;  
  propriété: valeur;  
}
```

- **Sélecteur** : identifie quoi styliser.
- **Propriétés** : attributs de style à modifier (ex. `color`, `font-size`).
- **Valeurs** : valeurs attribuées aux propriétés (ex. `red`, `16px`).

Sélecteur d'élément

Cible toutes les instances d'une balise HTML spécifique.

```
p { color: blue; }
```

Sélecteur de classe

Cible les éléments ayant un attribut `class`. Peut être réutilisé plusieurs fois dans la même page.

```
.intro { font-weight: bold; }
```

Sélecteur d'ID

Cible un élément unique avec un attribut `id`. À utiliser **une seule fois par page**.

```
#main-header { text-align: center; }
```

Grouping Selector

Applique les mêmes styles à plusieurs sélecteurs.

```
h1, h2, h3 { color: #438951; }
```

Propriétés courantes:

- **Couleurs & arrière-plans**
 - `color` : couleur du texte
 - `background-color` : couleur d'arrière-plan
 - Valeurs possibles : nommées (`red`), hexadécimales (`#438951`), ou RGB (`rgb(67, 137, 81)`)
- **Style du texte**
 - `font-family` : police (ex. `'Arial', sans-serif`)
 - `font-size` : taille du texte (ex. `16px`, `1.2em`)
 - `font-weight` : épaisseur (`bold`, `700`)
 - `text-align` : alignement (`left`, `center`)
- **Espacements & bordures**
 - `margin` : espace **extérieur** d'un élément
 - `padding` : espace **intérieur** d'un élément
 - `border` : bordure autour d'un élément

💡 **Exercice:** Prenez votre page **CV du jour 1**,

- Utilisez ces nouveaux concepts CSS pour :
 - changer les **couleurs**,
 - modifier les **polices**,
 - ajouter des **marges** et du **padding**.

👉 Faites-en une version **personnalisée et stylisée** !

Maîtriser les types d’affichage & Flexbox

Chaque élément HTML possède une propriété `display` par défaut qui détermine son comportement dans le flux du document. Comprendre ces types fondamentaux est essentiel pour organiser efficacement votre contenu.

Éléments inline

- S’écoulent avec le texte, en occupant uniquement la largeur nécessaire.
- On ne peut pas leur attribuer de largeur ou de hauteur, et les marges/espacements verticaux (*margin/padding*) sont ignorés.
- Exemples : `<span>`, `<a>`, `<strong>`.

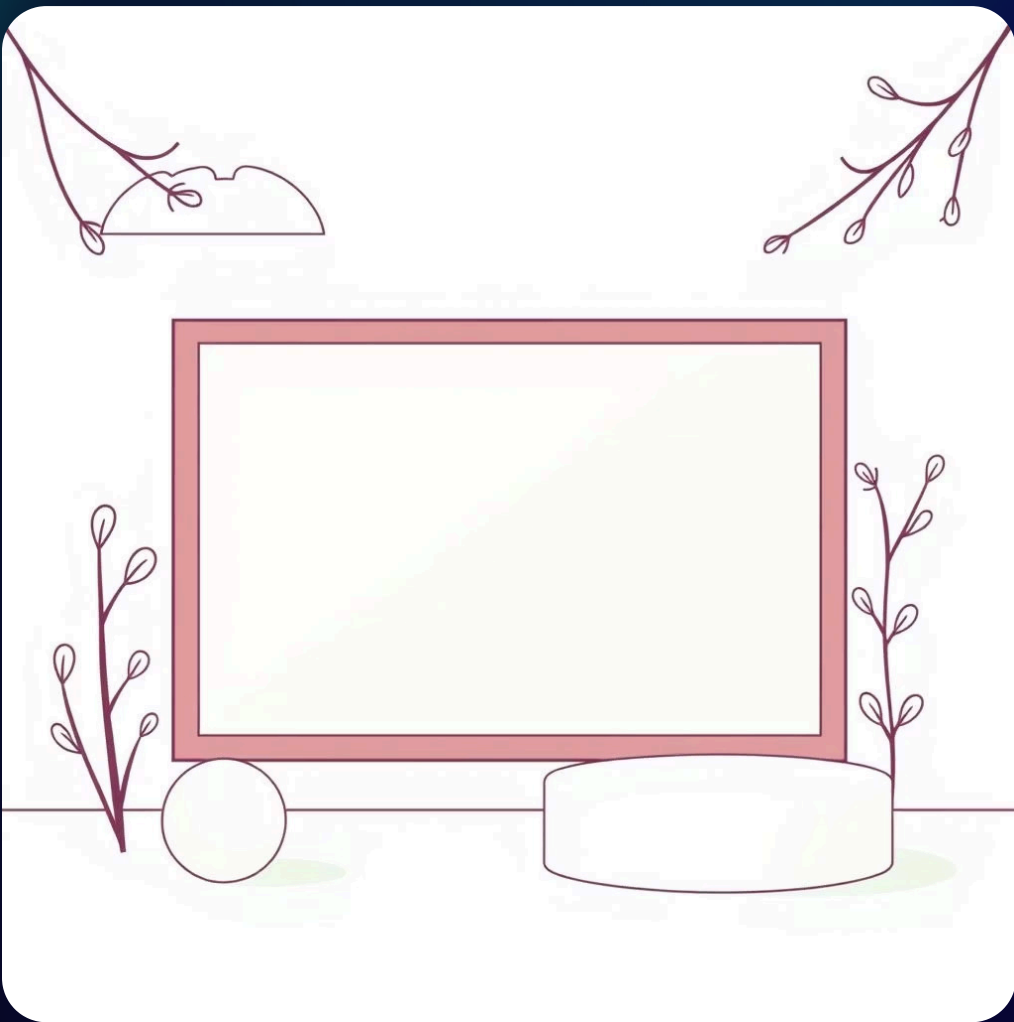
Éléments block-level

- Par défaut, ils occupent toute la largeur disponible et commencent sur une nouvelle ligne.
- Leur largeur, leur hauteur, ainsi que toutes les marges et espacements internes peuvent être définis.
- Exemples : `<p>`, `<div>`, `<h1>`.

Éléments inline-block

Un hybride : ils s’écoulent comme des éléments inline (ne forcent pas de saut de ligne),

- mais acceptent les propriétés des éléments block (largeur, hauteur, marges/espacements verticaux).
- Très utiles pour créer des rangées d’éléments avec des dimensions précises (ex. boutons, éléments de navigation).



Flexbox : un changement de paradigme pour la mise en page

Bien que les propriétés `display` traditionnelles soient essentielles, le design web moderne repose largement sur **Flexbox (Flexible Box Layout)** pour disposer efficacement des éléments en **une dimension** (ligne ou colonne).

👉 Flexbox est extrêmement puissant pour **aligner**, **distribuer** et **ordonner** le contenu.

Conteneur parent (Flex Container)

- Appliquez `display: flex;` sur l’élément parent.
- Ses enfants directs deviennent immédiatement des **éléments flexibles** (*Flex Items*).

Propriétés clés de Flexbox (sur le parent)

- `justify-content` : contrôle l’alignement des éléments sur l’axe principal (ex. `flex-start`, `center`, `space-between`).
- `align-items` : contrôle l’alignement des éléments sur l’axe secondaire (ex. `flex-start`, `center`, `stretch`).
- `flex-direction` : définit l’axe principal (`row`, `column`).



💡 **Demo:** Créons une barre de navigation horizontale dynamique avec **Flexbox**.

👉 Vous verrez à quel point il est facile d’**aligner** et d’**espacer** les éléments avec seulement quelques lignes de CSS !

Design réactif : s'adapter à chaque écran

Dans le monde multi-appareils d'aujourd'hui, vos sites web doivent être agréables à utiliser sur **ordinateurs**, **tablettes** et **smartphones**. C'est là qu'intervient le **responsive design** – une approche puissante qui permet de créer des mises en page flexibles s'adaptant parfaitement aux différentes tailles et orientations d'écran.

Pourquoi le responsive design ?

- Les utilisateurs accèdent aux sites web depuis une variété croissante d'appareils avec des dimensions d'écran différentes.
- Un site en largeur fixe conçu pour un ordinateur sera illisible sur un téléphone, entraînant une mauvaise expérience utilisateur.
- Le design réactif garantit **accessibilité** et **ergonomie** pour tous.

Unités relatives pour plus de flexibilité

- **% (pourcentage)** : relatif à la taille de l'élément parent.
- **em** : relatif à la taille de police de l'élément (ou de son parent).
- **rem** : relatif à la taille de police de l'élément racine `<html>`, offrant une base cohérente.
- **vw (viewport width)** : 1% de la largeur de la fenêtre d'affichage.
- **vh (viewport height)** : 1% de la hauteur de la fenêtre d'affichage.

👉 Utiliser ces unités plutôt que des pixels fixes permet aux éléments de **s'adapter proportionnellement** à la taille de l'écran.

Media Queries : le cerveau du responsive design

Les **media queries** en CSS permettent d'appliquer des styles différents en fonction des caractéristiques de l'appareil (largeur, hauteur, orientation, etc.).

Example Media Query

```
@media (max-width: 768px) {  
  body {  
    font-size: 14px;  
  }  
  .container {  
    flex-direction: column;  
  }  
}
```

Cette règle signifie : « *Si la largeur de l'écran est de 768 pixels ou moins, applique ces styles.* »

Idéal pour adapter les mises en page, les tailles de police ou masquer certains éléments sur mobile.

Mobile-First Design: Une approche moderne où l'on conçoit d'abord pour les **petits écrans** (mobile), puis on améliore progressivement le design pour les écrans plus grands grâce aux media queries.

👉 Cela assure une base solide pour tous les utilisateurs et conduit généralement à de meilleures performances sur mobile.

💡 **Demo:** Avec **Chrome DevTools**, nous pouvons simuler différentes tailles d'appareils.

Observez comment la page web s'adapte automatiquement lorsque nous changeons la fenêtre d'affichage, grâce au design réactif.

Ajouter JavaScript dans une page

Méthode interne :

```
<script>
  alert("Bonjour, bienvenue dans JavaScript !");
</script>
```

Méthode externe (meilleure pratique) :

```
<script src="script.js"></script>
```

Les bases du langage

- **Variables** : stocker des informations

```
let name = "Alice";
const age = 20;
```

- **Types de données** : texte (string), nombres (number), booléens (true/false).
- **Opérations simples** : addition, comparaison, concaténation de texte.

Interaction avec l'utilisateur

- **Alertes et saisies**

```
alert("Hello!");
let user = prompt("Quel est ton nom ?");
console.log("Bienvenue " + user);
```

Manipuler la page (DOM)

- Exemple : changer le texte d'un paragraphe.

```
<p id="demo">Texte initial</p>
<button onclick="changeText()">Changer</button>

<script>
  function changeText() {
    document.getElementById("demo").innerText = "Texte changé avec JS !";
  }
</script>
```

Exercices pratiques

- Créer une alerte au chargement de la page.
- Demander le prénom de l'utilisateur et l'afficher sur la page.
- Changer la couleur d'un paragraphe en cliquant sur un bouton.

Atelier pratique :

Styliser votre HTML & Construire une page d’accueil (Landing Page)

01	02	03
Style Your Day 1 Page (40 min) - Ouvrez votre page CV d’hier. - Créez un nouveau fichier CSS externe (ex. <code>style.css</code>) et liez-le à votre HTML. Tâches CSS : - Changez la couleur de fond de la page. - Stylisez les titres <code><h1></code>, <code><h2></code> avec des couleurs et polices différentes. - Ajoutez des espacements avec <code>margin</code> et <code>padding</code>. - Stylisez votre image : <pre>img { border-radius: 50%; border: 3px solid #438951; box-shadow: 2px 2px 8px rgba(0,0,0,0.2); }</pre>	Build a Simple Landing Page (1h 20 min) Structure HTML - Une barre de navigation <code><nav></code> avec des liens. - Une section "hero" avec un titre, un paragraphe et un bouton d’appel à l’action. - Un footer avec les informations de contact.	Objectifs CSS - Utiliser Flexbox pour aligner les liens de navigation horizontalement. - Centrer le contenu de la section hero avec Flexbox. - Ajouter un effet hover subtil sur le bouton. - Styliser le footer avec un fond sombre et un contraste de couleur.

JavaScript Interactivity

Maintenant que la structure et le style sont en place, ajoutons une couche d’interactivité.

Exercices JavaScript à intégrer

1. **Message de bienvenue au chargement de la page**

```
window.onload = function() {
  console.log("Bienvenue sur ma Landing Page !");
  alert("Bienvenue sur mon site 🚀");
};
```

1. **Interactivité du bouton CTA**

```
const btn = document.getElementById("cta-btn");
btn.addEventListener("click", () => {
  btn.innerText = "Merci d’avoir cliqué ✅";
  btn.style.backgroundColor = "#1e5631";
  alert("Action réussie !");
});
```

- Quand on clique, le texte change et un message apparaît.

1. **Changer le thème de la page** (bonus créatif ✨)

```
const themeBtn = document.createElement("button");
themeBtn.innerText = "Changer le thème";
document.body.appendChild(themeBtn);

themeBtn.addEventListener("click", () => {
  document.body.style.backgroundColor =
    document.body.style.backgroundColor === "white" ? "#f4f4f4" : "white";
});
```

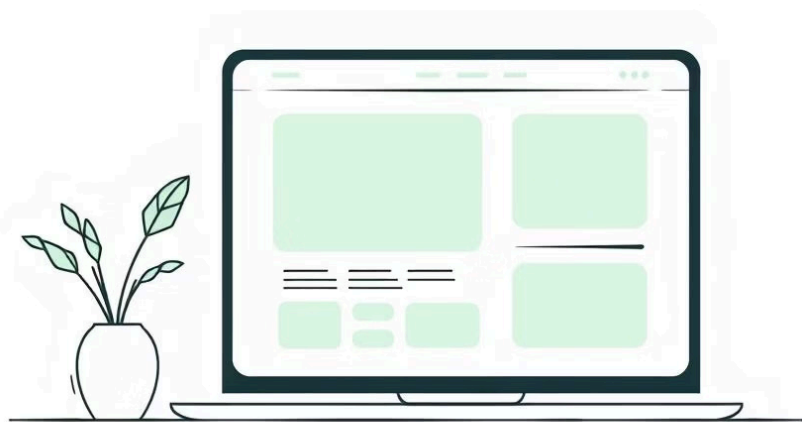
- Ajoutez un deuxième bouton qui change la couleur de fond.

Défi : Créez votre portfolio personnel & pensez responsive

Pour consolider votre maîtrise de la mise en page et du design adaptatif, votre prochaine mission consiste à créer **votre propre portfolio en ligne**. Cet exercice testera votre capacité à transformer une idée en un site fonctionnel en HTML, CSS et un peu de JavaScript, tout en respectant les principes de détail et d'adaptabilité.

Éléments à intégrer :

- **Navigation** : une barre claire avec votre logo ou nom et des liens de navigation (ex. Accueil, Projets, Contact).
- **Section Hero** : présentez vos informations personnelles (nom, rôle, description) et ajoutez vos projets principaux.
- **Section Deux Colonnes** : une partie texte (par ex. description d'un projet) et une image ou capture qui l'illustre.
- **Pied de page** : incluez vos coordonnées ou des liens vers vos réseaux sociaux.



Points clés à travailler :

- **Précision de la mise en page** : respectez les espacements, alignements et proportions. Utilisez **Flexbox** pour garder le contrôle.
- **Comportement responsive** : une fois la version bureau terminée, assurez-vous que le site s'adapte élégamment aux écrans plus petits.
- **Tests sur mobile/tablette** : utilisez les outils de développement du navigateur pour vérifier l'affichage.
- **Media Queries** : appliquez des règles `@media` pour que la section en deux colonnes se réorganise verticalement sur mobile.
- **Personnalisation CSS** : changez couleurs, polices et styles pour refléter votre identité.
- **Interaction JavaScript** : ajoutez 1 ou 2 petits scripts (par ex. un bouton qui affiche votre email, un effet de texte qui change au clic, ou un mini compteur).